Kharboosh Py

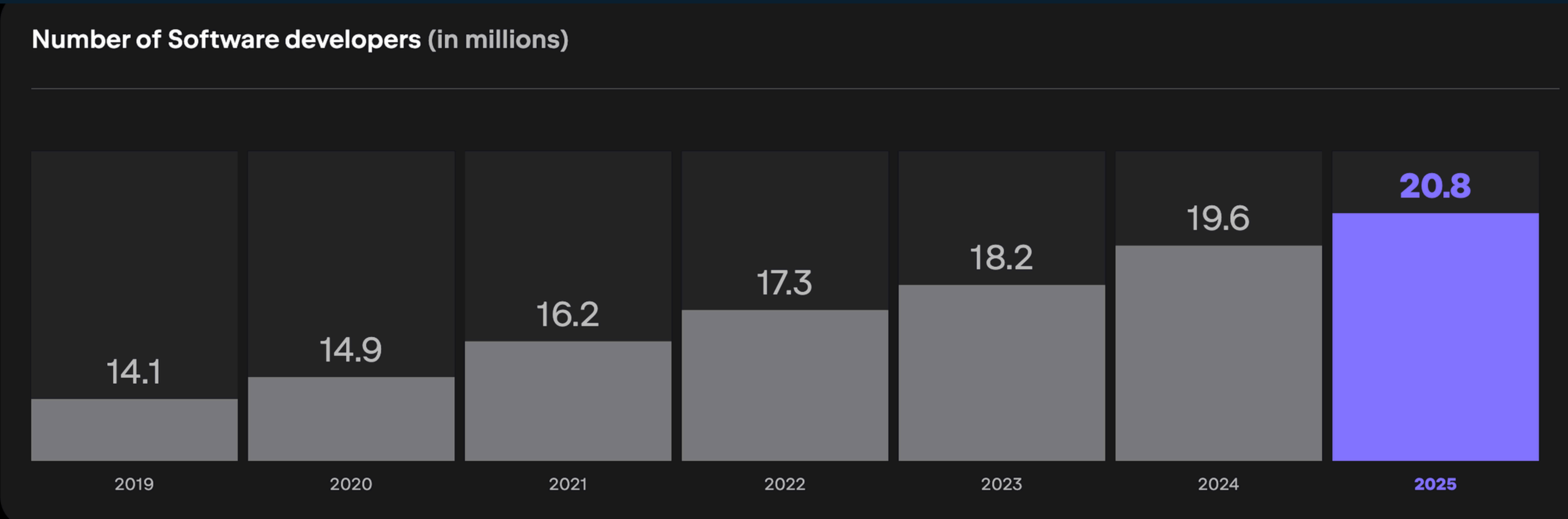
AI-Powered Code Review Platform

“Why do we still struggle to understand our own code even with AI tools?”

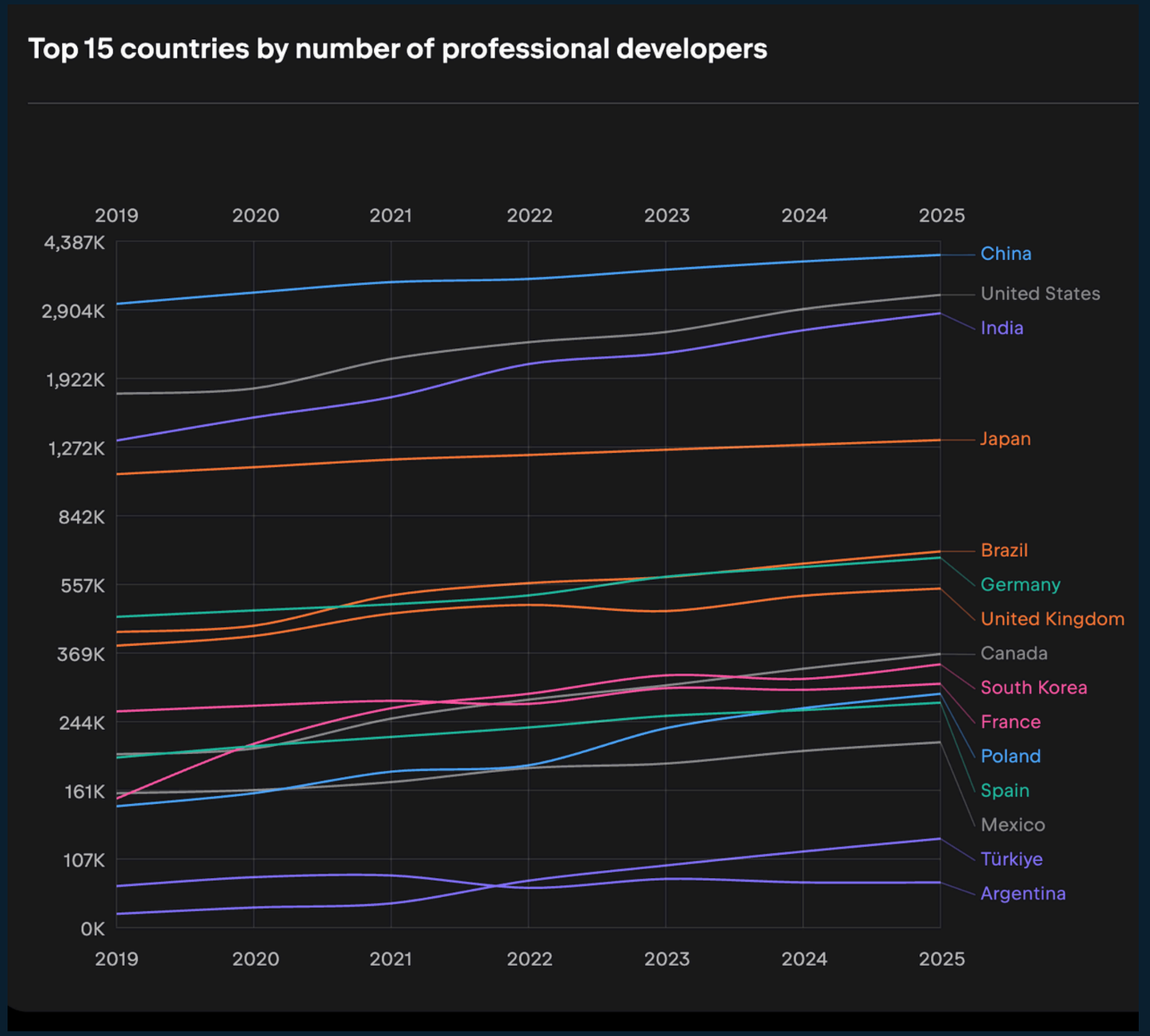
“If AI can write code for us, why can’t it fully understand and review it properly?”

“Have you ever spent hours debugging code only to realize the problem was a simple logic mistake?”

Number of Software developers



Top 15 countries by number of professional developers



Survey Insights

1	what's your current role?	How many years of programming experience	Which programming languages do you mainly	How do you usually work?	How do you currently review your code before
3	Student	3	Javascript	Individually	Reading it my self or by showing it to an AI ass
4	Junior Developer	4	Python, js	In a team	None
5	Junior Developer	5	C++ , Python	Individually	Read it twice or use gpt to confirm there is no
6	Junior Developer	6	C++, JS, Python	In a team	ChatGPT / instructors
7	Junior Developer	2	C++, Python	In a team	Myself
8	Junior Developer	0-1	ts - python	Individually	ai ide like antigravity
9	Student	4	Js c++ java	In a team	Claude
10	Junior Developer	5	Java	Individually	Chatgpt
11	Student	3	c++, python, javascript	In a team	Manually revising
12	Student	7	C++	In a team	Pull request through github
13	Junior Developer	0-1	Javascript, Python, C++	Individually	AI Tools like Claude
14	Junior Developer	3	Java JS Python	In a team	Ai does it
15	Student	4	Python, js	In a team	Via chaygpt, cluade
16	Junior Developer	1	c++, python	Individually	ai tools
17	Mid-level Senior	1	Java Spring - Python	In a team	Through github
18	Student	3	Python	Individually	By debugging

Survey Insights

1. User Profile

~90% Junior developers or students
~10% mid-level/senior developers
Majority work:
Individually (~50%)
In teams (~50%)

Programming Stack

Most used languages:
Python (very dominant)
JavaScript / TypeScript
C++
Java (less frequent but present)
Note: Strong focus on modern + academic + backend languages

3. Debugging Pain

~70% users take hours to days to detect bugs
~30% detect bugs in minutes only for simple issues
Majority report logic errors > syntax errors
Note: Logic bugs are the most expensive and time-consuming problem

Survey Insights

4. Tool Usage

~85–90% already use AI tools (ChatGPT, Claude, Copilot)
~60% still rely on manual review or teammates
Linters (ESLint/Pylint) used by ~40–50%
Note: AI adoption is high, but tools are not solving deep understanding problems

5. Main Pain Points

Most common issues:
Logic errors (highest frequency)
Code structure & readability
Lack of context in AI tools
Slow or inconsistent feedback from reviews
Over-reliance on AI causing confusion

6. Desired Features

Bug detection (100%)
Logic validation (very high demand)
Code explanation (very high demand)
Code improvement suggestions
Refactoring recommendations
Performance insights (some users explicitly want this)
Note: Developers want understanding + explanation, not just detection

7. Feedback Preferences

~80–90% prefer detailed feedback
~70% prefer real-time or IDE-based feedback
Accuracy is the #1 priority across almost all responses

8. Pricing Willingness

~70–80% are open to paying if value is clear
Preferred model:
Subscription-based (most common preference)
Some want freemium with limitations

The Problem with Tools

GitHub Copilot

Generates code without reviewing underlying logic, leaving developers susceptible to undetected errors in production.

ChatGPT

Provides general assistance without structured analysis, resulting in inconsistent code quality and rational explanations.

ESLint/Pylint

Focuses solely on syntax issues, lacking the reasoning capabilities needed to identify complex logical errors.

The Problem With Existing Tools

SonarQube

Often seen as **enterprise-heavy**, it lacks AI-driven explanations for logical errors in code reviews.

CodeRabbit

Focuses on **PR-based** feedback but fails to provide meaningful insights for junior developers.

Kharboosh Py

Combines AST, static analysis, and AI to offer **structured insights**, enhancing code quality and understanding.

Claude and the memory problem

Introducing Kharboosh Py

Kharbosh Py

Kharbosh Py is an AI-powered code review assistant designed to help developers understand their code, not just fix it.

It analyzes code beyond syntax by combining AST parsing, static analysis, and AI reasoning to detect bugs, explain logic, and suggest meaningful improvements.

Unlike traditional tools that only flag errors or generate code, Kharbosh Py focuses on deep understanding, clarity, and developer learning.

How It Works

1. Input Code

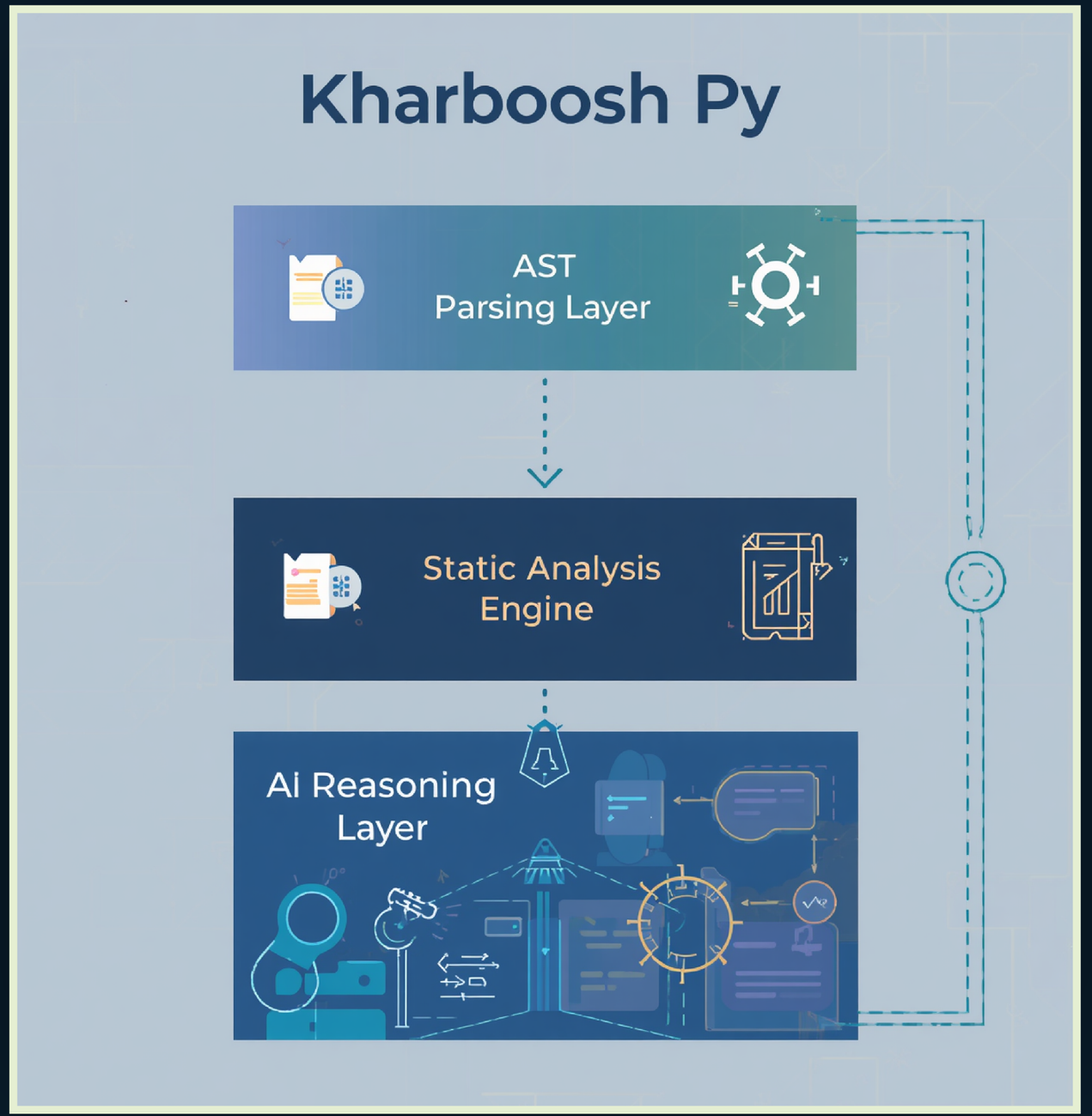
The developer writes or uploads code into the platform (web or IDE integration).

2. Code Parsing (AST Layer)

The system converts the code into an Abstract Syntax Tree (AST) to understand its structure and flow.

3. Static Analysis

A rule-based engine analyzes:
Syntax issues
Logical errors
Unreachable or inefficient code
Potential runtime risks



How It Works

4. AI Reasoning Layer

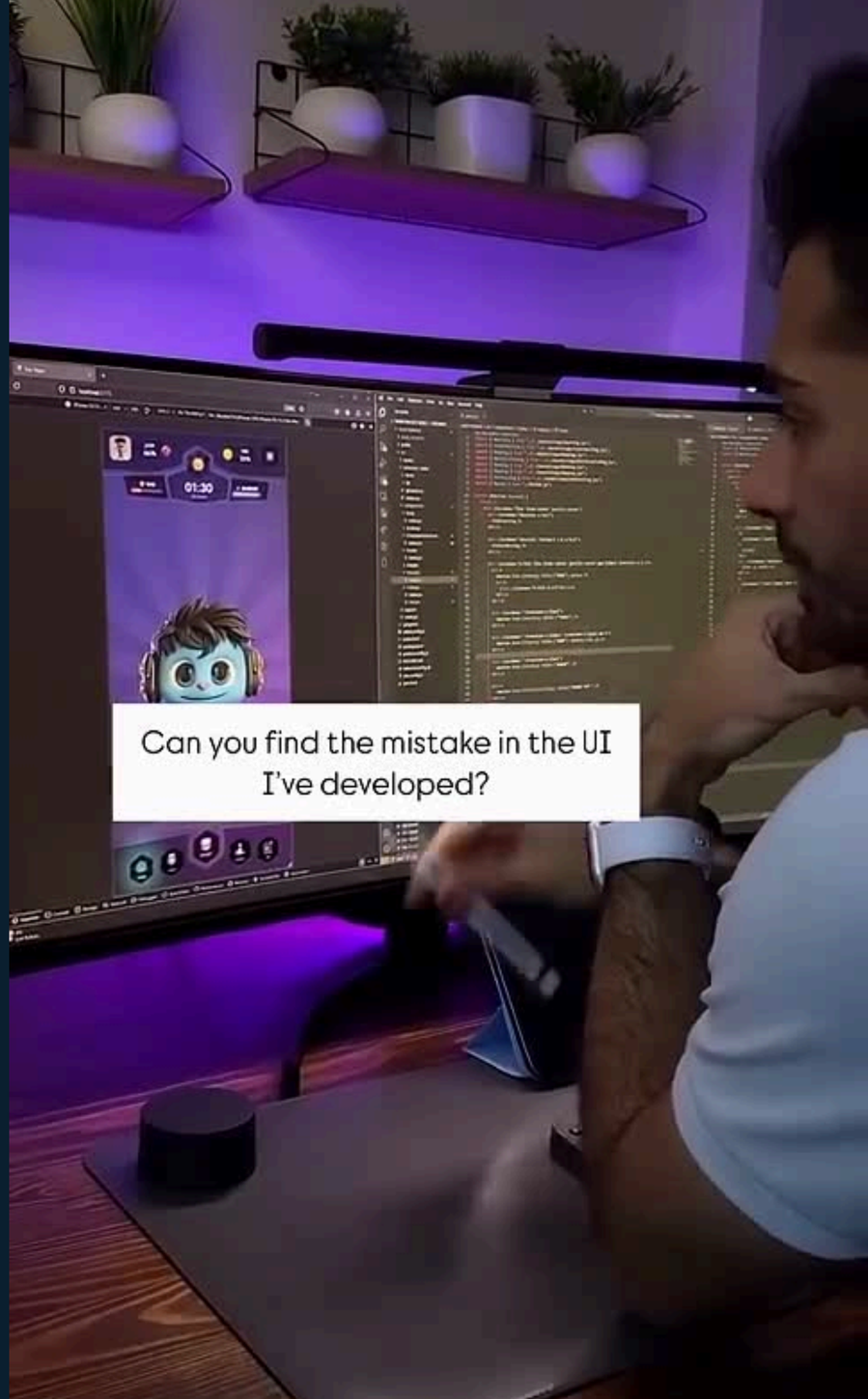
An AI model analyzes the code to:
Understand developer intent
Detect deeper logical problems
Suggest improvements and refactoring

5. Feedback Generation

All results are combined into one structured output:
Errors and bugs
Clear explanations
Improvement suggestions
Best practices and optimizations

6. Developer Output

The user receives:
Simple explanation of what the code does
What is wrong and why
How to fix and improve it
Optional performance insights (time & memory complexity)



MVP

https://kharboos
hpy.vercel.app

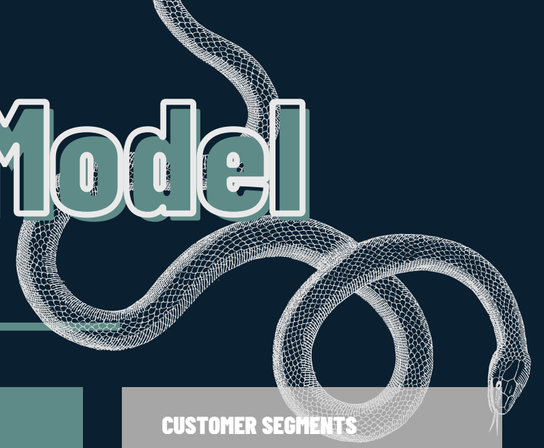
Business Market

SOM – Serviceable Obtainable Market
~$1M – $5M (early stage)
Based on:
10K – 50K users
$8/month pricing
Typical early capture = 0.1–2% of SAM
Meaning:
What we can realistically achieve in the first 1–3 years

TAM – Total Addressable Market
~$20B+
Based on the global developer tools & DevOps market (~$20B–$40B)
Includes:
code analysis tools
AI coding assistants
developer productivity tools
Meaning:
All developers globally who could use code review & AI debugging tools

SAM – Serviceable Addressable Market
~$5B – $8B
Subset of TAM:
Developers using modern stacks (Python, JS, AI tools)
Teams actively using code review & AI assistance
Typical SAM = 10–30% of TAM in SaaS
Meaning:
Developers we can realistically target with our product

Kharboosh Py Business Model



KEY PARTNERS

- Coding bootcamps and universities (early adoption + validation)
- Developer communities (growth and feedback)
- IDE platforms (e.g., VS Code Marketplace)
- AI providers (LLMs like Claude or similar models for reasoning layer)
- Educational creators and tech influencers

KEY ACTIVITIES

- Developing and improving analysis engine (AST + static + AI)
- Training and optimizing AI models
- Building integrations (VS Code, GitHub)
- Ensuring security and data privacy
- Continuous product improvement based on feedback
- Creating educational and marketing content

KEY RESOURCES

- AI models (LLMs for reasoning and explanation)
- Static analysis engine
- AST parsing system
- Code datasets (open-source + user feedback)
- Development team (engineering + AI expertise)
- Cloud infrastructure

VALUE PROPOSITIONS

- helps developers understand their code, not just fix it.
- Detects bugs beyond syntax using deep logic analysis
- Explains issues clearly, not just flags errors
- Combines AST parsing, static analysis, and AI reasoning in one system
- Improves code quality, readability, and maintainability
- Reduces debugging time and increases developer productivity
- Acts as a learning companion for junior developers
- Recommends best practices for clean, maintainable code
- Suggests refactoring opportunities and improved code structures
- Calculates time and memory complexity for performance optimization
- Identifies inefficient patterns and suggests better alternatives
- Provides step-by-step explanations to support learning and decision-making
- Provides structured benchmarking insights based on publicly available data
- Helps teams understand industry practices and identify improvement areas
- Built with strong privacy controls:
- Users choose what data is analyzed or shared
- No exposure of private or sensitive code
- Full control over visibility and data handling

CUSTOMER RELATIONSHIPS

- Self-service (easy onboarding and instant usage)
- Educational support (explanations and learning-focused feedback)
- Community engagement (developer communities and feedback loops)
- Continuous improvement based on user behavior and feedback
- Trust-based relationship through transparency and explainability

CHANNELS

- LinkedIn (professional reach)
- GitHub (developer ecosystem)
- YouTube (educational demos and tutorials)
- Developer communities (Discord, forums)
- VS Code Marketplace (future distribution)
- Bootcamps and universities (direct access to learners)

CUSTOMER SEGMENTS

- Junior developers and students
- Startup developers and small teams
- Coding bootcamps and instructors
- Software companies and enterprise teams

COST STRUCTURE

- AI and compute costs (API usage, model inference)
- Cloud infrastructure and hosting
- Development and engineering team
- Integration and maintenance (IDE, GitHub, CI/CD)
- Marketing and community building
- Security and compliance implementation

REVENUE STREAMS

- Subscription model (Freemium → Paid tiers)
- Pro plan ($8/month)
- Pro+ plan ($15/month)
- Team/Enterprise subscriptions ($10–15 per user/month)
- Future:
- Enterprise solutions (custom pricing)
- API access for integrations

Competitive Landscape

AI CODE ASSISTANTS:

GITHUB COPILOT

CHATGPT / CLAUDE

AMAZON CODEWHISPERER

→ FOCUS ON CODE GENERATION AND GENERAL ASSISTANCE, NOT DEEP LOGIC UNDERSTANDING

STATIC ANALYSIS & QUALITY TOOLS

SONARQUBE

CODEQL

SNYK CODE

→ DETECT BUGS, SECURITY ISSUES, AND CODE SMELLS USING PREDEFINED RULES

LINTERS (TRADITIONAL TOOLS):

ESLINT (JAVASCRIPT)

PYLINT (PYTHON)

→ FOCUS ON SYNTAX, FORMATTING, AND BASIC RULE ENFORCEMENT

AI CODE REVIEW TOOLS

CODERABBIT

CODIUMAI

→ AUTOMATED CODE REVIEWS USING AI, BUT LIMITED EXPLAINABILITY AND LEARNING DEPTH

Strategic Barriers

1. Technical Complexity
- Requires combining AST parsing, static analysis, and AI reasoning
- Hard to build a unified system with high accuracy and scalability and good memory

2. Data Dependency
- Needs high-quality and diverse code datasets
- Continuous learning required from real-world developer feedback

3. Trust & Adoption Barrier
- Developers are cautious about AI correctness in code
- Requires explainable AI to build reliability and confidence

4. Integration Barrier
- Must integrate into developer workflows (VS Code, GitHub, CI/CD)
- Requires seamless, non-intrusive user experience

5. Security & Privacy Barrier
- Handling sensitive or proprietary code safely
- No data retention by default
- Optional secure enterprise deployment

6. Competitive Pressure
- Competing with GitHub, OpenAI, Amazon
- Strong ecosystem lock-in from existing tools

7. Compute & Cost Barrier
- High cost of real-time AI + analysis processing
- Requires optimized hybrid architecture to reduce usage cost

Traction and Validation

Current Stage

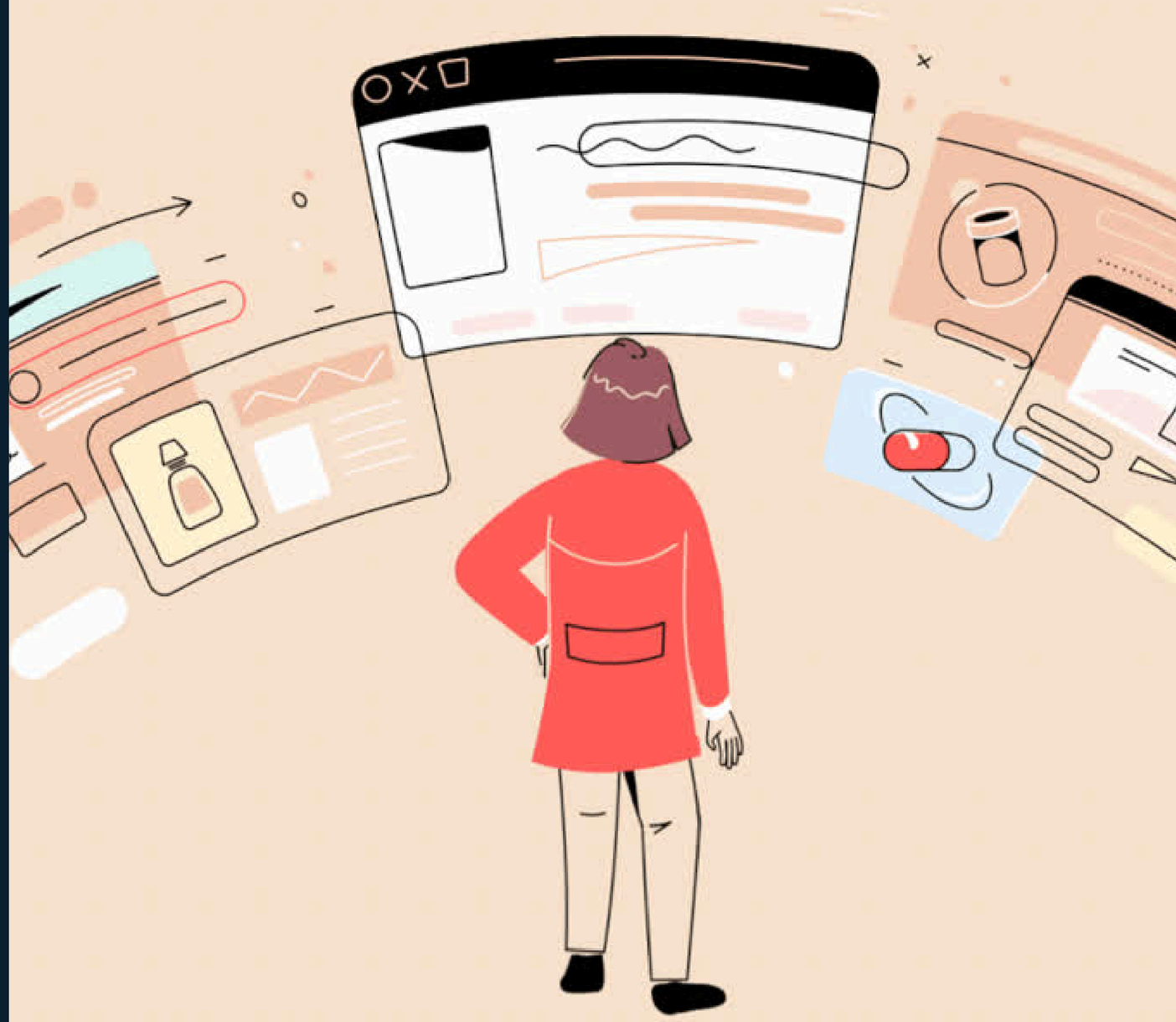
Problem validated through developer pain points
Survey conducted to understand user needs and preferences
Target users clearly identified (junior developers, teams, bootcamps)

MVP Validation

Build lightweight MVP (web-based code reviewer)
Focus on:
bug detection
logic explanation
basic AI suggestions
Test with one programming language (Python or JavaScript)

User Testing Plan

ITI students and university developers
Coding bootcamps
Developer communities (Discord / GitHub)
Collect feedback on:
usefulness
clarity of explanations
trust in results



Go-to-Market Strategy

1. Initial Focus

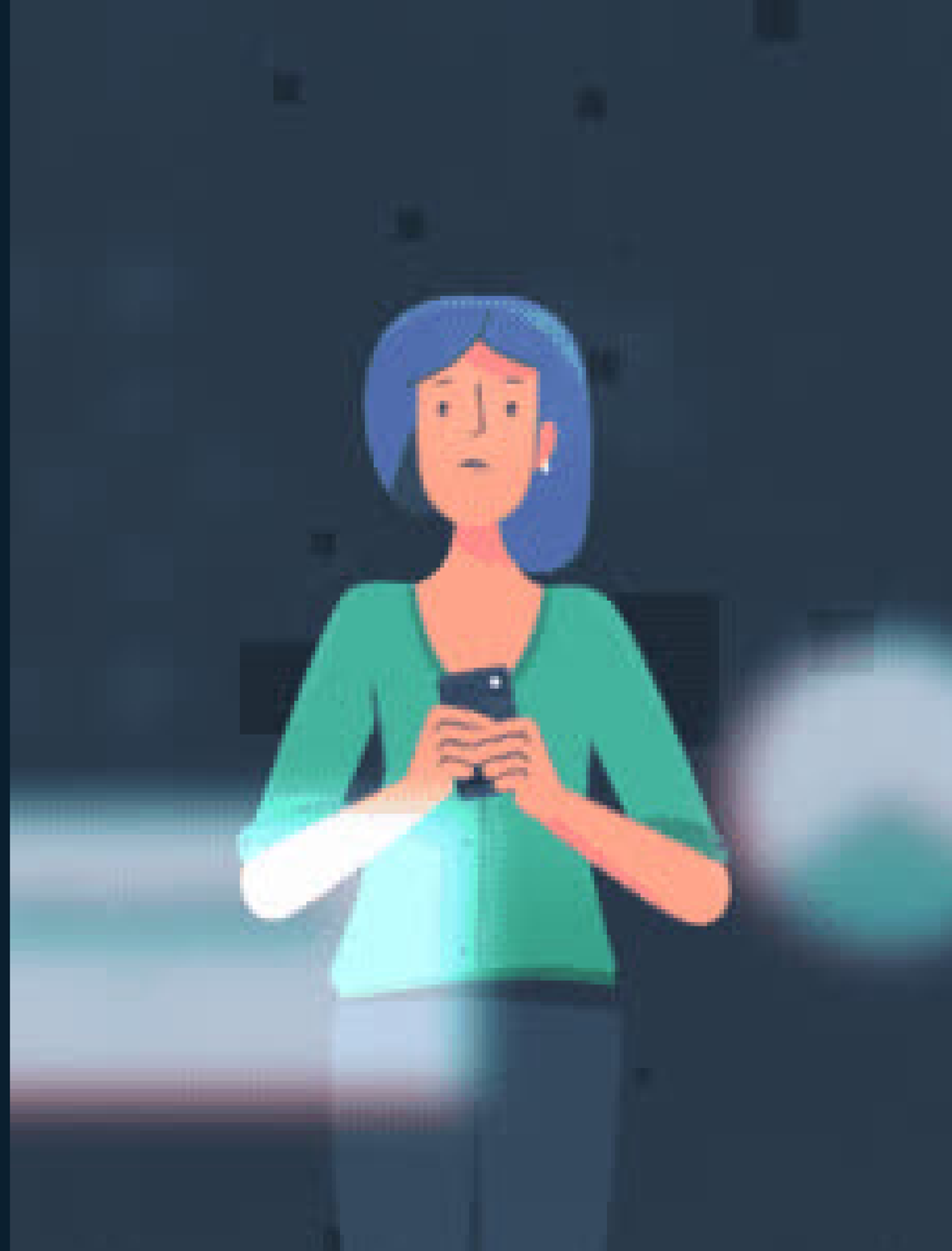
We will start with a narrow, high-impact entry:
Junior developers struggling with debugging
Students in bootcamps and universities
Python / JavaScript as first supported languages

2. Product-Led Growth

Users try the product instantly (no friction onboarding)
Free usage demonstrates real value through code analysis
Upgrade happens naturally when value is proven

3. Community-Driven Launch

Launch in developer communities (Discord, GitHub, Reddit)
Share real debugging cases and insights
Encourage early users to contribute feedback and improvements



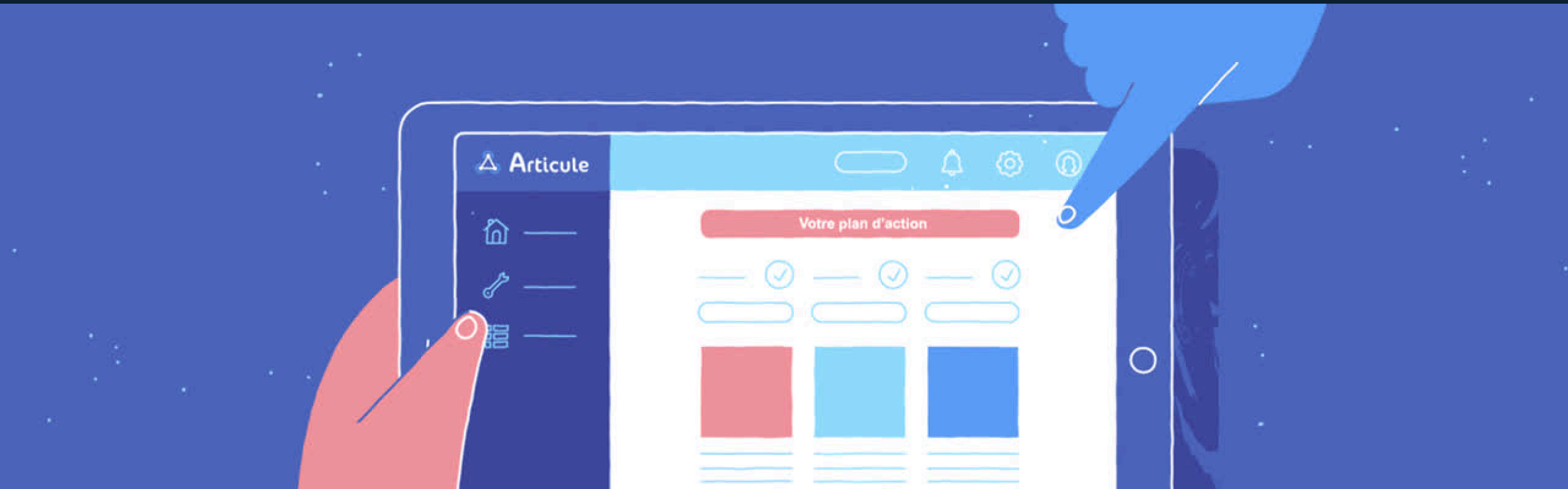
Cont. Go-to-Market Strategy

4. Content-Led Awareness

Educational posts about real coding mistakes
“Before vs After” code improvements
Short demo clips showing logic explanation in action
Focus: teach first, promote second

5. Early Validation Loop

Collect feedback directly from active users
Iterate rapidly based on real usage behavior
Use survey + in-app feedback to refine features



Security and Compliance

At Kharbosh Py, developer trust is a core priority.

Code Isolation: Each analysis runs in a secure, isolated environment
Zero Data Retention: User code is not stored after processing
Encryption: All data is protected using strong encryption standards
Privacy-First Design: User code is never used for model training
Compliance Ready: Designed to align with GDPR and modern data protection standards
User Control: Users can choose what data is shared or hidden at all times

Roadmap

Phase 1: MVP (0–3 months)

Web-based code input and analysis tool
Support for one language (Python or JavaScript)
Basic AST parsing and static analysis
Simple AI-generated explanations
Bug detection and logic error identification

Phase 2: Core Expansion (3–6 months)

Improved AI reasoning quality
Code refactoring suggestions
Best practices recommendations
Time and memory complexity analysis
Better UI/UX for developer workflow

Phase 3: Developer Integration (6–9 months)

VS Code extension
GitHub integration (PR code review)
Real-time feedback while coding
Team collaboration features

Phase 4: Intelligence Scaling (9–12 months)

Multi-language support
Personalized feedback based on developer behavior
Advanced debugging insights
Performance optimization suggestions at scale

Phase 5: Ecosystem Expansion (Future Vision)

API for companies and SaaS integration
Enterprise version with advanced security controls
Learning mode for bootcamps and education platforms
AI-powered coding mentor system

Meet Our Team

Rana Salem

Shahd Ramadan

Shahd Mostafa

Menna Mohamed

Rehab Hamed

Ahmed Ehab

Ahmed Wael

Ibrahim Elsayed

Khalid Cherif

Get in touch with us today!

EMAIL

FOLLOW US

PHONE